학습데이터처리

데이터 전처리/Dataframe 응용

© KOPO-Dept.
AI Engineering Limited

데이터 전처리

- 누락 데이터(결측치) 처리
  - 데이터프레임의 원소 데이터 값이 종종 누락되는 경우가 있음
  - 유효한 데이터 값이 존재하지 않는 누락 데이터를 NaN(Not a Number)으로 표시
  - ML 모델을 만들기 전에 **반드시 누락 데이터를 제거하거나 다른 적절한 값으로 대체하는 과정**이 필요
    - 누락데이터가 많아지면 **데이터의 품질 및 모델의 성능이 떨어지고**, ML 분석 알고리즘을 왜곡하는 현상이 발생할 수도 있음

  - 누락데이터 확인
    - df.info()를 통해 유효한 데이터(non-null, 즉 NaN 값이 아닌)의 개수 확인

```
In [3]: df.info()
Out[3]:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
survived       891 non-null int64
pclass         891 non-null int64
sex            891 non-null object
age            714 non-null float64
sibsp          891 non-null int64
```

데이터 전처리

- 누락 데이터(결측치) 처리
  - 누락데이터 확인
    - df.info()를 통해 유효한 데이터(non-null, 즉 NaN 값이 아닌)의 개수 확인
    - 특정 열을 기반으로 value_count 사용: df['deck'].value_count(dropna=False)
      - Info()에서 deck 열에 누락데이터가 제일 많았음
    - isnull(), notnull() 활용: 어떤 원소가 누락인지를 true/false로 반환

```
NaN    688
C       59
B       47
D       33
E       32
A       15
F       13
G        4
Name: deck, dtype: int64
```

- **isnull()** : 누락 데이터면 True를 반환하고, 유효한 데이터가 존재하면 False를 반환한다.
- **notnull()** : 유효한 데이터가 존재하면 True를 반환하고, 누락 데이터면 False를 반환한다.

	survived	pclass	sex	age	...	deck	embark_town	alive	alone
0	False	False	False	False	...	True	False	False	False
1	False	False	False	False	...	False	False	False	False
2	False	False	False	False	...	True	False	False	False
3	False	False	False	False	...	False	False	False	False

  - df.isnull().sum(): **각 열에서 NaN인 개수 반환**

데이터 전처리

- 누락 데이터(결측치) 처리
  - 누락데이터 처리
    - 행, 열 삭제(제거)
      - drop, dropna 메서드 사용
        - » Drop(colums=[' deck’]
        - » Dropna(axis=1, thresh=500)
        - » Dropna(subset=[‘age’], axis=0)
    - NaN ➔ 다른 값으로 대체
      - fillna 메서드 사용 (※ NaN이 아니라 0, -, ? 등의 문자가 있을 시에는 replace 메서드 사용: df['horsepower'].replace('?', np.nan, inplace=True))
        - » NaN의 데이터를 제거하지 않고, 평균 값으로 대체
        - » mean_age = df[‘age’].mean()
        - » df[‘age’].fillna(mean_age)
      - ffill, bfill 사용
        - » df[‘embard_town’].fillna(method=‘ffill’)

데이터 전처리

- 중복 데이터 처리
  - 중복 데이터 확인
    - duplicated() 메서드 사용
      - 전에 나온 행들과 비교하여 중복되는 행이면 True, 아니면(처음 나오는 행이면) False

```
2
3  # 라이브러리 불러오기
4  import pandas as pd
5
6  # 중복 데이터를 갖는 데이터프레임 만들기
7  df = pd.DataFrame({'c1':['a', 'a', 'b', 'a', 'b'],
8                     'c2':[1, 1, 1, 2, 2],
9                     'c3':[1, 1, 2, 2, 2]})
10 print(df)
11 print('\n')
12
13 # 데이터프레임 전체 행 데이터 중에서 중복값 찾기
14 df_dup = df.duplicated()
15 print(df_dup)
16 print('\n')
```

```
  c1  c2  c3
0  a   1   1
1  a   1   1
2  b   1   2
3  a   2   2
4  b   2   2


0    False
1     True
2    False
3    False
4    False
dtype: bool
```

```
18 # 데이터프레임의 특정 열 데이터에서 중복값 찾기
19 col_dup = df['c2'].duplicated()
20 print(col_dup)
```

```
0    False
1     True
2     True
3    False
4     True
Name: c2, dtype: bool
```

데이터 전처리

- 중복 데이터 처리
  - 중복 데이터 삭제
    - drop_duplicated() 메서드 사용
      - subset 옵션 사용 가능 drop_duplicated(subset=[“col1, col2,..,”])

```
12
13  # 데이터프레임에서 중복 행 제거
14  df2 = df.drop_duplicates()
15  print(df2)
16  print('\n')
```

```
   c1  c2  c3
0  a   1   1
1  a   1   1
2  b   1   2
3  a   2   2
4  b   2   2


   c1  c2  c3
0  a   1   1
2  b   1   2
3  a   2   2
4  b   2   2
```

```
18  # c2, c3열을 기준으로 중복 행 제거
19  df3 = df.drop_duplicates(subset=['c2', 'c3'])
20  print(df3)
```

```
   c1  c2  c3
0  a   1   1
2  b   1   2
3  a   2   2
```

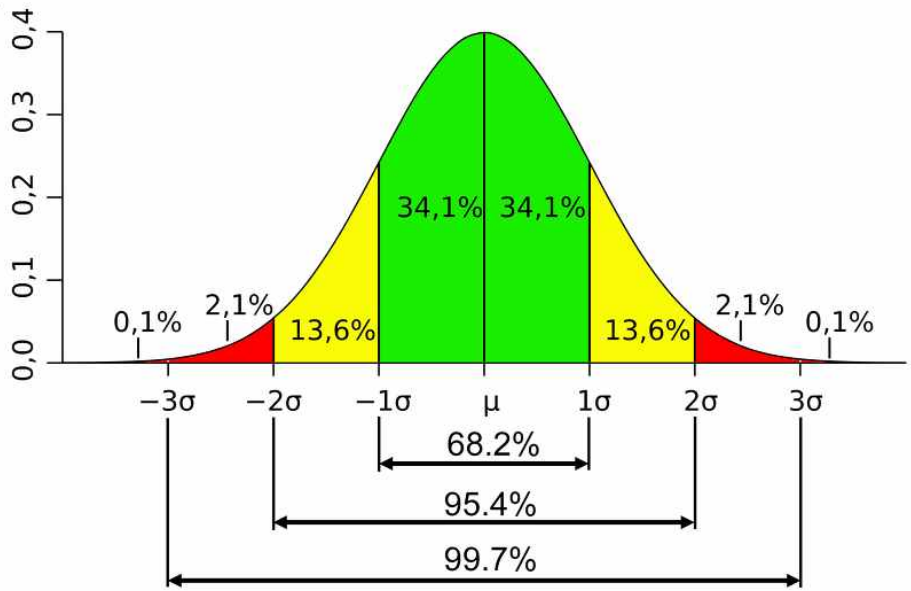
데이터 전처리

- 이상치 데이터 처리
  - Z-Score 사용 등
    - Z-Score: 평균으로 부터 얼마나 떨어져 있는지를 표준편차를 이용해 설명하는 척도
      - 예: z-score가 1.67이면 1.67표준편차만큼 평균값에서 멀어져 있다는 것.
      - 예: -2이하, +2이상 데이터는 삭제

$$z = \frac{x - \mu}{\sigma}$$

μ = Mean

σ = Standard Deviation



정규분포

© KOPO-Dept.
AI Engineering Limited

데이터 전처리

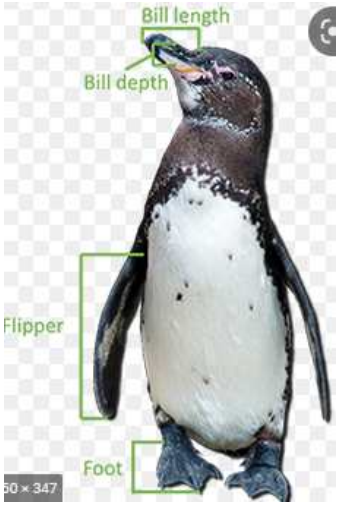
- 실습

```python
import pandas as pd
import seaborn as sns  # Load Dataset

pd.set_option('display.max_columns', None)
df_titanic = sns.load_dataset('titanic')
# print(df.head())
print(df_titanic.info())
print()
...
print(df_titanic["deck"].value_counts(dropna=False))
...
print(df_titanic.head().isnull())
```

seaborn 에서 기본적으로 제공하는
타이타닉, 펭귄 데이터셋 활용

1. 누락 데이터 확인
2. 불필요한 데이터 삭제 등



© KOPO-Dept.
AI Engineering Limited

데이터 전처리

- 데이터 표준화
  - 단위 환산
    - 외국 데이터를 가져오면 국내에서 잘 사용하지 않는 도량형 단위를 사용하는 경우가 많음
      - 예: 영미권에서 주로 사용하는 마일, 야드 등을 한국에서 사용하는 미터, 평수 등으로 변경
      - auto-mpg.csv 파일의 mpg 컬럼을 이용하여 kpl (km per liter)로 변경

```
4  import pandas as pd
5
6  # read_csv() 함수로 df 생성
7  df = pd.read_csv('./auto-mpg.csv', header=None)
8
9  # 열 이름 지정
10 df.columns = ['mpg','cylinders','displacement','horsepower','weight',
11              'acceleration','model year','origin','name']
12 print(df.head(3))
13 print('\n')
14
15 # mpg(mile per gallon)를 kpl(kilometer per liter)로 변환(mpg_to_kpl = 0.425)
16 mpg_to_kpl = 1.60934/3.78541
17
18 # mpg 열에 0.425를 곱한 결과를 새로운 열(kpl)에 추가
19 df['kpl'] = df['mpg'] * mpg_to_kpl
20 print(df.head(3))
21 print('\n')
```

1마일은 1.60934km, 1갤론은 3.78541리터

데이터 전처리

- 데이터 표준화
  - 자료형 변환
    - 컬럼의 속성을 분석하여 필요시 자료형을 변환하는 것이 좋음 (숫자가 문자열(object)로 저장된 경우, 숫자형(int or float)으로 변경 필요
      - 예: horsepower(? 문자등으로 인해 해당 컬럼의 데이터가 object임)는 숫자형으로 변경 필요
      - 예: model year, origin은 카테고리를 나타내는 범주형이 적절
    - **astype('자료형')** 사용
      - df['horsepower'] = df['horsepower'].astype('float')
      - df['origin'] = df['origin'].astype('category')

```
# 누락 데이터('?') 삭제
import numpy as np
df['horsepower'].replace('?', np.nan, inplace=True)      # '?'을 np.nan으로 변경
df.dropna(subset=['horsepower'], axis=0, inplace=True)   # 누락 데이터 행 삭제
df['horsepower'] = df['horsepower'].astype('float')      # 문자열을 실수형으로 변환

# horsepower 열의 자료형 확인
print(df['horsepower'].dtypes)
```

데이터 전처리

- 범주형(카테고리) 데이터 처리
  - 구간 분할
    - 연속 데이터를 그대로 사용하기 보다는 일정한 구간(bin)으로 나눠서 분석하는 것이 효율적인 경우가 있음
    - 가격, 비용, 효율 등 연속적인 값을 일정한 수준이나 정도를 나타내는 이산적인 값으로 표현
    - Binning(구간분할): 연속 변수를 일정한 구간으로 나누고, 각 구간을 범주형 이산 변수로 변환
      - 예: horsepower를 숫자로 표시하는 대신 '저출력, 보통출력, 고출력' 구간으로 나누어서 표시
      - Pandas: cut 메서드 활용하기 위해 먼저 구간을 나누어야 함
        - » 1) numpy의 histogram() 함수 활용, bin을 3개(low, normal, high)로 구간을 나눔
        - » 2) pandas의 cut 메서드에 histogram() 반환 값(경계값 리스트) 사용 등



 © KOPO-Dept.
AI Engineering Limited

데이터 전처리

- 범주형(카테고리) 데이터 처리
  - 구간 분할 (계속)

```
14  # horsepower 열의 누락 데이터('?') 삭제하고 실수형으로 변환
15  df['horsepower'].replace('?', np.nan, inplace=True)        # '?'을 np.nan으로 변경
16  df.dropna(subset=['horsepower'], axis=0, inplace=True)     # 누락 데이터 행 삭제
17  df['horsepower'] = df['horsepower'].astype('float')        # 문자열을 실수형으로 변환
18
19  # np.histogram 함수로 3개의 bin으로 나누는 경계값의 리스트 구하기
20  count, bin_dividers = np.histogram(df['horsepower'], bins=3)
21  print(bin_dividers)
```

```
[ 46.      107.33333333  168.66666667  230.        ]
```

bin_dividers

```
23  # 3개의 bin에 이름 지정
24  bin_names = ['저출력', '보통출력', '고출력']
25
26  # pd.cut 함수로 각 데이터를 3개의 bin에 할당
27  df['hp_bin'] = pd.cut(x=df['horsepower'],          # 데이터 배열
28                        bins=bin_dividers,          # 경계값 리스트
29                        labels=bin_names,           # bin 이름
30                        include_lowest=True)        # 첫 경계값 포함
31
32  # horsepower 열, hp_bin 열의 첫 15행 출력
33  print(df[['horsepower', 'hp_bin']].head(15))
```

	horsepower	hp_bin
0	130.0	보통출력
1	165.0	보통출력
2	150.0	보통출력
3	150.0	보통출력
4	140.0	보통출력
5	198.0	고출력
6	220.0	고출력

데이터 전처리

- 범주형(카테고리) 데이터 처리
  - 더미 변수
    - 연속 데이터를 범주형 데이터로 변환하였지만, 회귀분석 등 머신러닝 알고리즘에 적용하기 위해서는 컴퓨터가 인식 가능한 입력 값으로 변환해야 함
    - Low: 1, Normal: 2, High: 3 와 같이 처리하는 것보다는
    - One-hot Encoding 방식이 적합 (0, 1로 어떤 특성이 있는지 없는지 여부만을 표시)
      - pandas: get_dummies() 함수 사용

```
30
31  # hp_bin 열의 범주형 데이터를 더미 변수로 변환
32  horsepower_dummies = pd.get_dummies(df['hp_bin'])
33  print(horsepower_dummies.head(15))
```

	저출력	보통출력	고출력
0	0	1	0
1	0	1	0
2	0	1	0
3	0	1	0
4	0	1	0
5	0	0	1
6	0	0	1
7	0	0	1
8	0	0	1
9	0	0	1
10	0	0	1

© KOPO-Dept.
AI Engineering Limited

데이터 전처리

- 정규화
  - 각 변수(컬럼, Feature)에 들어 있는 숫자 데이터의 상대적 크기 차이 때문에 머신러닝 결과가 달라질 수 있음.
    - A 변수: 0~1000 범위의 값
    - B 변수: 0~1 범위의 값
      - 이 경우, 상대적으로 큰 숫자 값을 갖는 A변수의 영향이 더 커짐.
  - 숫자 데이터의 상대적인 크기 차이를 제거(정규화)할 필요 있음
    - 각 열에 속하는 데이터 값을 동일한 크기 기준으로 나눈 비율로 변환 (0~1 사이 또는 -1~1 사이 값으로 변환)
      - 각 열의 데이터를 해당 열의 최대값(의 절대값)으로 나누는 방법 등

```
22
23  # horsepower 열의 최대값의 절대값으로 모든 데이터를 나눠서 저장
24  df.horsepower = df.horsepower/abs(df.horsepower.max())
```

```
25
26  print(df.horsepower.head())
27  print('\n')
28  print(df.horsepower.describe())
```

```
0    0.565217
1    0.717391
2    0.652174
3    0.652174
4    0.608696
Name: horsepower, dtype: float64


count    392.000000
mean       0.454215
std        0.167353
min        0.200000
25%        0.326087
50%        0.406522
75%        0.547826
max        1.000000
Name: horsepower, dtype: float64
```

학습데이터처리

Dataframe 응용

© KOPO-Dept.
AI Engineering Limited

DataFrame 응용

- 함수매핑
  - 시리즈 또는 데이터프레임의 개별 원소를 특정 함수에 일대일 대응시키는 과정
  - 사용자 함수(lambda 함수포함) 적용 가능: 판다스 기본 함수로 처리하기 어려운 연산을 데이터프레임 등 판다스 객체에 적용하는 것이 가능

  - 1) **개별 원소**에 함수 매핑
    - 시리즈 원소에 함수 매핑
      - 시리즈 객체에 apply() 메서드 적용
      - 원소 한개씩 가지고 와서 함수 적용 (결과 리턴: 시리즈 객체의 원소 수만큼)

```
시리즈의 원소에 함수 매핑: Series객체.apply(매핑 함수)
```

        » sr1 = df['age'].apply(add_10) #add_10: 사용자 함수
    - 데이터프레임 원소에 함수 매핑
      - applymap() 메서드 활용

```
데이터프레임의 원소에 함수 매핑: DataFrame 객체.applymap(매핑 함수)
```

        » df_map = df['age', 'fare'].applymap(add_10) #add_10: 사용자 함수

DataFrame 응용

- 함수매핑
  - 시리즈 또는 데이터프레임의 개별 원소를 특정 함수에 일대일 대응시키는 과정
  - 사용자 함수(lambda 함수포함) 적용 가능: 판다스 기본 함수로 처리하기 어려운 연산을 데이터프레임 등 판다스 객체에 적용하는 것이 가능

  - 2) **시리즈 객체**에 함수 매핑
    - 데이터프레임의 **각 열(행)에 함수 매핑**
      - 데이터프레임에 apply() 메서드 적용
      - 시리즈를 한개씩 가지고 와서 함수 적용
        - » 사용자 함수가 시리즈를 입력받아 시리즈를 반환하는 함수: 데이터프레임 반환
        - » 사용자 함수가 시리즈를 입력받아 하나의 값을 반환하는 함수: 시리즈 반환

```
데이터프레임의 열에 함수 매핑: DataFrame 객체.apply(매핑 함수, axis=0)
```

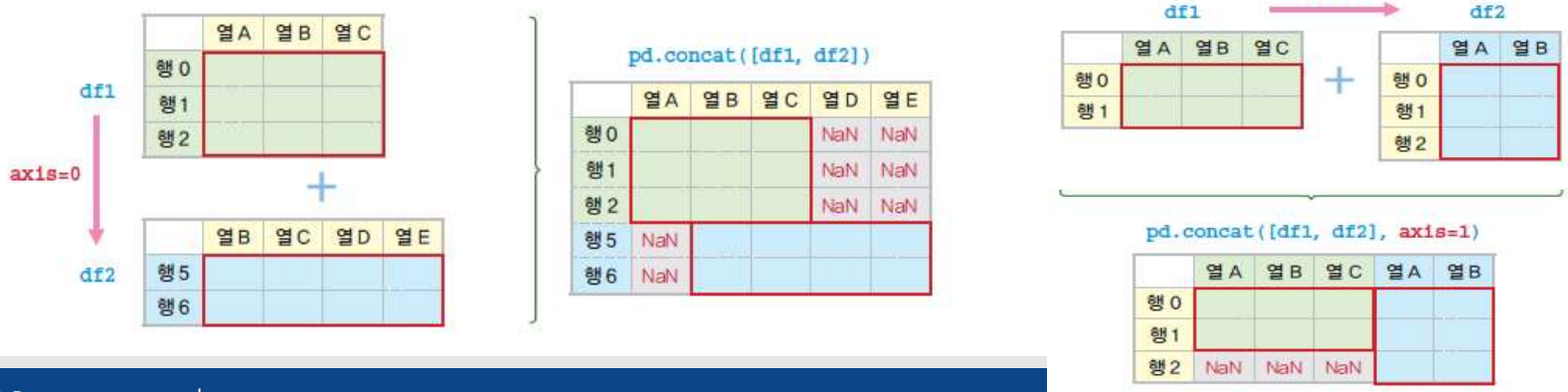
```
데이터프레임의 행에 함수 매핑: DataFrame 객체.apply(매핑 함수, axis=1)
```

DataFrame 응용

- 데이터프레임 합치기
  - 1) 데이터프레임 **연결**
    - 기존 데이터프레임의 형태를 유지하면서 이어 붙이는 개념
    - concat() 메서드 사용: **pd.concat([df1, df2])**

```
데이터프레임 연결: pandas.concat(데이터프레임의 리스트)
```

  - 옵션
    - » axis(0: row 방향으로 연결, 1: col 방향으로 연결)
    - » join(outer: 중복상관없이 연결, inner: 중복되는 데이터만 연결)
    - » ignore_index (True: 합칠때 각각의 Dataframe 인덱스를 유지 X, 새로 0부터 부여)

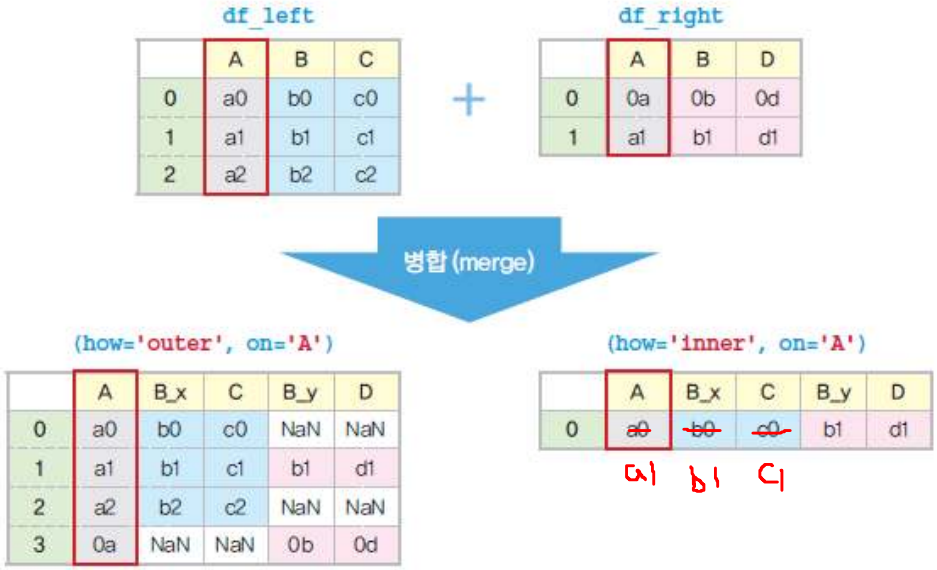


DataFrame 응용

- 데이터프레임 합치기
  - 2) 데이터프레임 **병합**
    - SQL join() 명령과 유사한 방식으로 어떤 기준에 의해 두 데이터 프레임을 병합하는 개념
      - 기준이 되는 열이나 인덱스를 키라고 부르며, 키가 되는 열이나 인덱스는 반드시 양쪽 데이터프레임에 존재해야 함
    - Merge() 메서드 사용: **pd.merge(df1, df2)**

```
데이터프레임 병합: pandas.merge( df_left, df_right, how='inner', on=None )
```

Default



df_left

	A	B	C
0	a0	b0	c0
1	a1	b1	c1
2	a2	b2	c2

df_right

	A	B	D
0	0a	0b	0d
1	a1	b1	d1

(how='outer', on='A')

	A	B_x	C	B_y	D
0	a0	b0	c0	NaN	NaN
1	a1	b1	c1	b1	d1
2	a2	b2	c2	NaN	NaN
3	0a	NaN	NaN	0b	0d

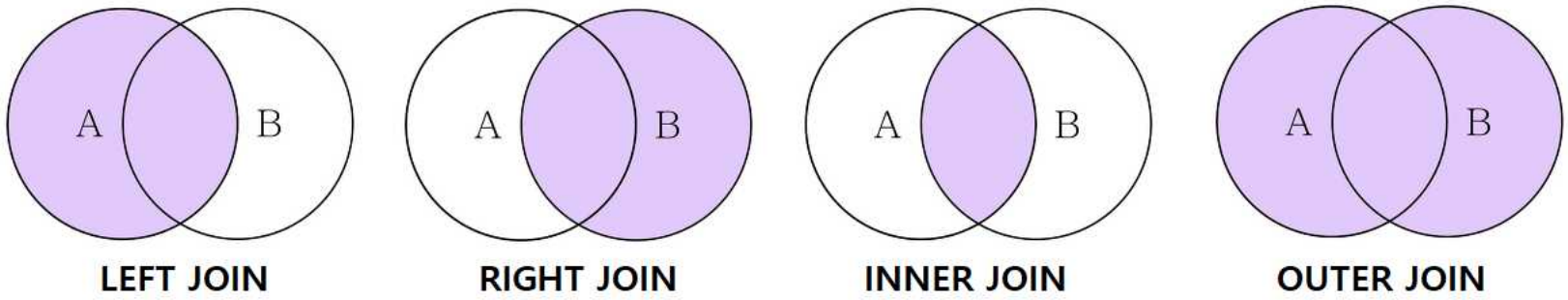
(how='inner', on='A')

	A	B_x	C	B_y	D
0	~~a0~~	~~b0~~	~~c0~~	b1	d1

a1 b1 c1

DataFrame 응용

- 데이터프레임 합치기
  - 2) 데이터프레임 **병합**
    - SQL join() 명령과 유사한 방식으로 어떤 기준에 의해 두 데이터 프레임을 병합하는 개념
      - 기준이 되는 열이나 인덱스를 키라고 부르며, 키가 되는 열이나 인덱스는 반드시 양쪽 데이터프레임에 존재해야 함
    - Merge() 메서드 사용: **pd.merge(df1, df2)**
      - 옵션
        - » on=None: 두 데이터프레임에 공통으로 속하는 모든 열을 기준(키)으로 병합 (left_on : 기준열 이름이 다를 때, 왼쪽 기준열, right_on : 기준열 이름이 다를 때, 오른쪽 기준열)
          - on='컬럼명'
        - » how(inner: 기준이 되는 열의 데이터가 양쪽 데이터프레임에 공통으로 존재하는 교집합일 경우에만 추출 outer:기준되는 열의 데이터가 데이터프레임 중 어느 한쪽에만 있더라도 데이터를 병합)



DataFrame 응용

- 그룹연산
  - 1) 그룹 객체 만들기
    - groupby() 메서드 사용: 데이터프레임의 특정 열을 기준으로 데이터프레임을 분할하여 **그룹 객체를 반환**
    - 기준이 되는 열은 1개도 가능하고, 여러 열을 리스트로 입력 [멀티인덱스 생성됨] 할 수 있음

```
그룹 연산(분할): DataFrame 객체.groupby(기준이 되는 열)
```

```
그룹 연산(분할): DataFrame 객체.groupby(기준이 되는 열의 리스트)
```

		age	fare	survived
class	sex			
First	female	34.611765	106.125798	0.968085
	male	41.281386	67.226127	0.368852
Second	female	28.722973	21.970121	0.921053
	male	30.740707	19.741782	0.157407
Third	female	21.750000	16.118810	0.500000
	male	26.507589	12.661633	0.135447

  - 2) 그룹 연산
    - 데이터 집계: 판다스 기본 내장함수에는 mean(), max(), min(), sum(), count(), std(), etc.

```
표준편차 데이터 집계(내장 함수): group 객체.std()
```

    - 그룹 객체 필터링: filter() 메서드를 적용할때 조건식을 가진 함수를 전달하면 조건이 참인 그룹만을 남김.

```
그룹 객체 필터링: group 객체.filter(조건식 함수)
```


© KOPO-Dept.
AI Engineering Limited

DataFrame 응용

- 실습

```python
import numpy as np
import pandas as pd
import seaborn as sns #dataset 로드

# titanic 데이터셋에서 age, fare 2개 열을 선택하여 데이터프레임 만들기
titanic = sns.load_dataset('titanic')
df = titanic.loc[:, ['age', 'fare']]
df['ten'] = 10
print(df.head())
print('\n')


# 사용자 함수 정의
def add_10(n):  # 10을 더하는 함수
    return n + 10


def add_two_obj(a, b):  # 두 객체의 합
    return a + b


print(add_10(10))
print(add_two_obj(10, 10))
```


© KOPO-Dept.
AI Engineering Limited

Thank you

Q&A

www.kopo.ac.kr
jsshin7@kopo.ac.kr

 © KOPO-Dept.
AI Engineering Limited